

50.051 Programming Language Concepts (Y2026)

Authored by: Matthieu de Mari & Dileepa Fernando, January 2026.

Important. This course handout serves as the single source of reference for *anything* that is related to this course. All updates will be reflected in this document.

Contents

1. About this Course
 - Learning Objectives
 - Measurable Outcomes
 - Assessment
 - Midterm exam Schedule
 - Final exam Schedule
 - Course Policy: Failing to Follow Instructions
 - Course Policy: Academic Honesty
 - Course Policy: Medical Certificates
 - Course Policy: Use of Large Language Models
 - Course Policy: Late Submission Policy
2. Getting Ready for the Course
 - Mac OS
 - Linux
 - Windows
 - Common Compiler for Practical Assignments Submissions
 - Setting up the flags on OnlineGDB
 - Programming tips with the flags
3. Instructor Office Hours
4. Course Schedule & Deliverables
5. Lessons
 - Part I: Introduction to C and C++
 - Part II: Introduction to Compiler Engineering Concepts
6. Readings
7. Course Assessment

Practical Assignments (20%)

1. Practical 1: C and Binary Operation
2. Practical 2: Pointers and Strings
3. Practical 3: FSM and RegEx
4. Practical 4: Tokenization and Context Free Grammars
5. Practical 5: C++
6. Practical 6: Top-Down Parsing

Group Project (for a total of 20%)

Project Presentation in Week 13 (5% of the 20% above)

Code (10% of the 20% above)

Project Report (5%)

Peer-Review (X%)

Midterm Exam (20%)

Final Exam (30%)

Participation (8%)

Course Feedback (2%)

8. A note for auditing students

1. About this Course

In this module, students will learn about data representation, programming and compilation. In the first part, we will learn how binary is used to encode values (data) and instructions (code). We will then learn how to create complex data types and how object oriented programming works behind the scenes. In the second part, we will work on the different aspects of compilation, including lexical and syntactic analysis, parsing and programming language design. Students will complete four small projects (2 individual and 2 group assignments) about memory management in C, C++, and compilation (lexical and syntactic analysis and parsing). By gaining this knowledge, students will be able to understand how programming languages are designed.

Learning Objectives

1. Program in C and C++
2. Design a compiler for an imperative programming language
3. Understand how data/instructions are stored and represented in the computer's memory
4. Understand the link between imperative and object-oriented languages
5. Understand the compilation process

Measurable Outcomes

1. Understand how memory is used to store data and instructions on a computer
2. Ability to program in C and C++
3. Understand the imperative programming paradigm
4. Understand the compilation process
5. Ability to learn any imperative programming language

Assessment

Practical Assignments - One in Week 1 - One in Week 3 - One in Week 5 - One in Week 8 - One in Week 10 - One in Week 12	20% 2.5% 5% 2.5% 2.5% 5% 2.5%	Grade based on <i>individual</i> performance
Group Project - Project Briefing in Week 2 - Project Presentation in Week 13 - Code to be submitted in Week 13 - Final Report to be submitted in Week 13	20% - 5% 10% 5%	Grade based on <i>team</i> performance
Class Participation - In-class activity, discussion, attendance - Feedback	10% 8% 2%	Grade based on <i>individual</i> performance
Exams - Midterm Exam in Week 6 - Final Exam in Week 14	50% 20% 30%	Grade based on <i>individual</i> performance

Midterm exam Schedule

The current MidTerm Exam date is set to W8 (no longer W6 as initially announced).
 The date is now March 18th, 11.45am. More details in the Assessment section below.

Final exam Schedule

XXX

Commented [MD1]: TBC

Course Policy: Failing to Follow Instructions

Failure to adhere to written instructions for assignments, project, midterms, final exams, or any guidelines outlined in this course handout **may result in a penalty** to your overall grade.

Course Policy: Academic Honesty

We take plagiarism seriously and we will take strict measures against students who plagiarise the work of others. If it is found that you have plagiarized in your submissions, all parties involved will receive an immediate zero for that assignment for your first offence. If you unwisely choose to repeat this, you will immediately fail the course.

Course Policy: Medical Certificates

- There will be no make-up quizzes/exams of any sort.
- If you have a valid MC covering the date of the assessment, as per the norm, you must apply for an LOA with the university.
- We will decide on an appropriate means of grading you.
- Without an MC and LOA from the university, you will receive zero for the assessment.
- We only accept one short term MC in the duration of the course. If you produce a second MC for a second assessment, **you will receive a zero for the assessment**. We will only accept a second MC if it pertains to you being hospitalised/quarantined/seriously ill.

Course Policy: Use of Large Language Models

- Large Language Models **cannot and should not be used to generate code that you use in your project or lab**.
- You may use LLMs to fix/improve your writing for the project report and make the message smoother but **LLMs cannot be used to generate the original text instead of you**.
- If you use LLM for copywriting, please acknowledge it in the report and explain how you used it.

Course Policy: Late Submission Policy

A late submission penalty will be applied to late submission (assignments or group assignment) as follows:

- Between 0 and 6 hours late: -25%
- 6 to 12 hours late: -50%
- 12 to 18 hours late: -75%
- Above 18 hours late: -100%

2. Getting Ready for the Course

To get ready for the course, make sure you can compile C/C++ code on your PC.

To do so, you have to install gcc or another compiler. [Take note of the requirements for the common compiler for practicals.](#)

Mac OS

1. Open a terminal
2. Type "gcc"
 - a. If the output is "clang: error: no input files" it means gcc is already installed
 - b. If a pop-up appears and asks you to install Xcode tools, please follow the procedure
3. You're good to go!
Note: the compiler is **clang**, not gcc (gcc is just an alias for clang)

Linux

1. **gcc** is installed by default!

Windows

1. Option 1: Install [Visual Studio Code](#)
2. Option 2: Install [Cygwin](#)
3. Option 3: Install [MinGW](#)

Common Compiler for Practical Assignments Submissions

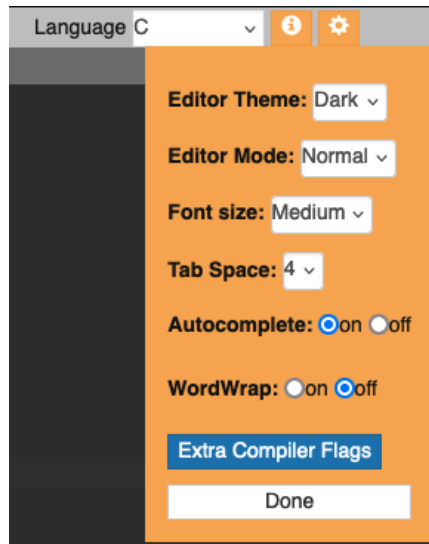
For all six practical assignments, your code will need to compile with this online compiler: [OnlineGDB](#). It may happen that some specific behavior (e.g. forgetting an empty line at the end of a file) may lead to compilation error with one compiler but not others, even with the exact same flags.

Each assignment will have a set of compilation flags that you will have to use. If your code does not compile with the compiler mentioned above, and all the required flags, you will lose points for this assignment. In case of discrepancy, the online compiler will be used to determine whether your code is correctly compiling or not.

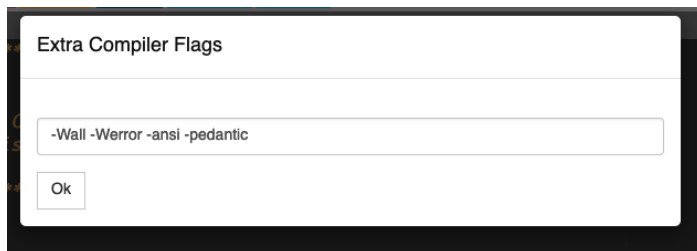
Setting up the flags on OnlineGDB

To add the required flags (e.g. `-Wall -Werror -ansi -pedantic`), locate the cog icon on the top right corner of the website. Click on it, then click on the blue button "Extra Compiler Flags".

Restricted



Then insert the flags in the text box of the popup that appears:



Programming tips with the flags

As the flags we will be using in the first few weeks are rather "strict", here are [some tips](#) on the common error messages and mistakes and how to avoid them.

Restricted

3. Instructor's Info and Office Hours

Professor: Matthieu de Mari (matthieu_demari@sutd.edu.sg)

Matthieu De Mari received the PhD degree in Mathematics and Computer Science from CentraleSupélec, France, in July 2015. He was involved with several research projects using various tools from statistics, simulation, optimization, and machine learning theories, to analyse the performance of the future generation of wireless networks. More specifically, his research and teaching interests revolve around game theory, mean field game theory, deep learning, and reinforcement learning.

- Please contact me via email to arrange a meeting time first, as I will be teaching several courses and conducting Capstone projects during this term.
- Online or Physical (2.401.07), make an appointment via email before (will be teaching this course and the Deep Learning one as well!).

Professor: Dileepa Fernando (dileepa_fernando@sutd.edu.sg)

Dileepa Fernando is a Lecturer in the Information Systems Technology and Design pillar at the Singapore University of Technology and Design (SUTD). Dileepa completed his Bachelor of Science in Engineering (BSc Eng) with a specialization in Computer Science and Engineering at the University of Moratuwa, Sri Lanka. Later, he obtained a PhD in Computer Science from the National University of Singapore, with a focus on utilizing model checking to ensure the game theory security of probabilistic protocols. Expanding his doctoral work, he led the secure machine learning project at HP-NTU corporate lab. Following his passion of teaching he started his teaching career in Sri Lanka Technological Campus (SLTC) and joined SUTD in 2023. His teaching experience in SUTD covers most parts of computer science including Software Engineering, Cyber Security and Artificial Intelligence. Outside his professional life, he enjoys public speaking and holds the position of vice president membership at the Breakthrough Toastmasters Club, Singapore. He spends his leisure time singing and playing the guitar.

- Please contact me via email to arrange a meeting time first, as I will be teaching several courses and conducting Capstone projects during this term.
- Online or Physical (1.402.24), make an appointment via email before (will be teaching this course and the Deep Learning one as well!).

Teaching Assistant: Juan David Salazar Rodríguez (1008372@mymail.sutd.edu.sg)

Juan David Salazar Rodríguez is a PhD candidate in the Architecture and Sustainable Design pillar at the Singapore University of Technology and Design (SUTD), originally from Colombia. He holds a Bachelor's degree in Civil Engineering from the National University of Colombia and a Master's degree from National Cheng Kung University (NCKU) in Taiwan. His research investigates artificial intelligence as a collaborative agent in architectural education, focusing on large language models, AI personas, and mixed reality systems to support design critique, reflection, and creative learning in studio-based pedagogy. His work sits at the intersection of human-AI interaction, design computing, and educational technology. Outside academia, he enjoys football and tennis, is a devoted Real Madrid supporter, and serves as the SUTD tennis team captain.

4. Course Schedule & Deliverables

Week 1 to Week 13 will be done **physically**, in class (venue 1.410) as currently planned.
 The detailed (tentative) schedule can be found below.

Classes on green background happen in class, whereas classes in purple background fall on Public Holiday. **Announcements will be made regarding these public holidays, but likely to be replaced by online lectures, with self-study to be done by students outside of class time.**

Note on instructors symbols used in timetable:

*: This session is taught by Prof Dileepa

^: This session is taught by Prof Matthieu

	Lesson 1	Lesson 2	Lesson 3
Week 1 (27 Jan)	Intro to C*	Instructions*	Practical 1: Types, Binary Operations*
Week 2 (3 Feb)	Arrays & Structs*	Structs & Pointers*	Pointers Practice + Project Briefing*
Week 3 (10 Feb)	Strings*	Memory Segments*	Practical 2: Pointers & Strings*
Week 4 (17 Feb)	Files*	Libraries*	Compilation in C*
Week 5 (24 Feb)	Finite State Machines^	Regular Expressions^	Finite State Machines Practice and Practical 3 FSM/RegEx^
Week 6 (3 Mar)	Static, Union, Misc*	Intro to Compilation^	Tokenization 1^
Week 7 (10 Mar)	RECESS WEEK		
Week 8 (17 Mar)	Tokenization 2^	More about Parsing and Practical 5: Tokenization and CFG announcement^	MidTerm*^
Week 9 (24 Mar)	Context Free Grammars (CFGs)^	CFG & Syntax Directed Translation ^	Top-Down Parsing ^
Week 10 (31 Mar)	C++*	C++*	Practice in C++ and Practical 4 C++*
Week 11 (7 Apr)	Bottom-Up Parsing^	Semantic Analysis^	Semantic Analysis^

Restricted



Week 12 (14 Apr)	Intermediate Representations and Code Generation^	IR Optimisation and Middle-end^	Practical 6: Top-Down Parsing^
Week 13 (21 Apr)	Conference	Project Presentations*^	Project Presentations*^
Week 14 (28 Apr)	Final Exam*^		

Please note that on W13S1, there will be a conference, replacing the lecture. More details about the conference will be shared at a later date. Please note that attendance for this conference is mandatory, and is accounted for in your participation score at the end of the term.

5. Lessons

The course is split in two parts, and materials will be uploaded weekly on eDimension.

Part I: Introduction to C and C++

1. Intro to C
2. Instructions
3. Computer Memory
 - a. Binary
 - b. Turing machine
4. Data Representation (simple types)
 - a. Binary & Bitwise operations
 - b. Integers (integer overflow)
 - c. Floating point numbers (floating point problem)
 - d. Characters
5. Data Representation (Arrays)
 - a. Arrays [known size] (buffer overflow)
 - b. Strings
 - c. Lists [of any size]
 - d. n-D arrays
 - e. Structures
 - f. The infamous case of **void ***
 - g. Functions
6. Pointers & Memory Management
 - a. Memory allocation
 - b. Freeing Memory (Double free)
 - c. Dangling pointers
7. Files
8. Processes

Restricted

- a. General concepts
 - b. Memory Segmentation
- 9. Basic Libraries
- 10. Terminal and Debugging
 - a. Makefile
 - b. Valgrind
- 11. Pre-processing in C
 - a. Macros
- 12. C++: object oriented programming
 - a. "Struct + functions + checks $\approx$ object
 - b. Polymorphism
 - c. Virtual functions
 - d. Operator overloading
- 13. Other:
 - a. Compilation process
 - b. Vulnerabilities, Buffer overflow

Part II: Introduction to Compiler Engineering Concepts

- 1. Pre-processing & Lexical Analysis
 - a. Finite State Machines and Regular Expressions
 - b. Tokenization
- 2. Lexical Analysis
 - a. Context Free Grammars
 - b. Using CFGs for syntactic validity
- 3. Syntactic Analysis Parsing
 - a. Top-Down Parsing
 - b. Bottom-Up Parsing
- 4. Semantic Analysis
- 5. Intermediate Representation and Code Generation
- 6. Compiler Middle-End and Code Optimization
- 7. Compiler Backend and linking to 50.002 Computation Structures

6. Readings

The following textbooks are used as resources supporting this course.
 Feel free to have a look at them.

- 1. SEI CERT C Coding Standard, Rules for Developing Safe, Reliable, and Secure Systems
- 2. C Programming Language, (Kernighan & Ritchie)
- 3. A Complete Guide to Programming in C++, (Kirch-Prinz & Prinz)
- 4. Introduction to Compilers and Language Design (Thain, chapters 1-8)

7. Course Assessment

Practical Assignments (20%)

There will be six practical assignments, on Weeks 1, 3, 5, 8, 10, 12 usually started or done during the third session of the week. Labs are individual (not group-based) and can be evaluated either by check-off during the session or be longer.

For assignments not fully done in class, you will need to submit the code on eDimension two weeks later, on Wednesday (the day before session 3), at 23:59.

1. Practical 1: C and Binary Operation
Date: Instructions given during Week 1.
2. No submission required, just checkoff done in class during session.
3. Practical 2: Pointers and Strings
Date: Instructions given in class on Week 3.
Submit on eDimension by 26 Feb, 11.59pm.
4. Practical 3: FSM and RegEx
Date: Start in class on Week 5.
No submission required, just checkoff done in class during session.
5. Practical 4: Tokenization and Context Free Grammars
Date: Instructions given in class on Week 8, and uploaded on eDimension.
Submit on eDimension by **2nd April, 11.59pm.**
6. Practical 5: C++
Date: Instructions given in class on Week 10.
7. Submit on eDimension by **17th April, 11.59pm.**
8. Practical 6: Top-Down Parsing
Date: Done in class on Week 12.
No submission required, just checkoff done in class during session.

Group Project (for a total of 20%)

Groups will be done once, and will be the same for both Part 1 and Part 2 of the course.
For the group project, you will have to tackle a larger problem in scope.

During Week 2 or Week 3, we will present the instructions for the project. Groups will also be created in Week 2-3 and will remain the same throughout the semester. Students will be free to create their own groups of up to 4 members. Use the link on eDimension to submit your group information. A small project proposal will be required, and the final submission of the project is expected in Week 12. More details will be announced in class and on eDimension.

Project Presentation in Week 13 (5% of the 20% above)

During the last week of class, every group will summarize their project in a **15 minutes** presentation during Session 2 and Session 3. More details will follow at a closer date.
Please submit the slides on eDimension (if any).

Project Code (10% of the 20% above)

Please submit the code in C, in a Zipfile, with a Makefile.
Note: the deadline for the code and report are the same.

Project Report (5%)

Your group will need to submit a final report on the project, detailing the process of development, how the project works, the individual contribution of each group member...
The report does not have a page limit.
Note: the deadline for the code and report are the same.

Peer-Review (X%)

There will be a peer-review component. As a student, you will be asked to provide feedback on the performance of each of your group members. The feedback from the other members will be aggregated and converted into a coefficient between 0 and 1, that will be multiplied by the group grade to compute the individual grade of each student.

For example: if your group got 18/20 for the project and your personal coefficient is 0.8, your own grade will be 14.4/20.

Midterm Exam (20%)

1. **Date: March 19th 2026, 11.45am**, 1 hour long exam. Location: 1.409.
2. Will be conducted **in person**, on **paper**.
3. Questions format will be multiple choice, short and long answers (e.g. writing C code).
4. This will be a **closed book** exam, testing concepts from W1-5 and W6S1 Lesson.
Introduction to Compilers lecture and the Tokenization 1 lecture will not be tested.
5. You will be allowed to bring a **cheat sheet** as per university policy:
 - a. **One** A4 page (21x29.7 cm)

Restricted



- b. **Hand-written (with your hands and a regular pen/pencil, not printed, even if you wrote on a tablet)**
- c. **Double-sided**
- d. To clarify: it's one page **overall**, not one page per part. See above for the requirements
- e. No calculator or any other electronic device.

Final Exam (30%)

- 6. **Date: 30th April 2026, 9-11am.** Duration: 2 hours. Location: 2.607.
- 7. Will be conducted **in person**, on **paper**.
- 8. Questions format will be multiple choice, short and long answers (e.g. writing C code).
- 9. This will be a **closed book** exam.
- 10. You will be allowed to bring a **cheat sheet** as per university policy:
 - a. **One A4 page (21x29.7 cm)**
 - b. **Hand-written (with your hands and a regular pen/pencil, not printed)**
 - c. **Double-sided**
 - d. To clarify: it's one page **overall**, not one page per part. See above for the requirements
 - e. No calculator or any other electronic device.

Participation (8%)

Includes attendance and participation in class activities. Attendance may not be taken every class. In case you are not sure if attendance has been taken, ask the professor or the TA.

Course Feedback (2%)

Fill the survey at the end of the semester.

8. On the use of Generative AI tools

Generative AI tools, including LLMs such as ChatGPT, Claude, Gemini, Copilot and similar systems, may be used in this course as learning and productivity tools.

Students may use them to clarify concepts, debug code, understand error messages, improve writing, generate small examples, or brainstorm project ideas. However, they must not be used to replace the student's own understanding, reasoning, implementation, or analysis.

In particular:

- ✓ You may use Generative AI tools to support your learning.
- ✓ You may use them to improve the clarity or structure of your work.
- ✓ You may use them to help debug or better understand your code.

Restricted

Restricted



X You may not submit AI-generated answers, code, reports, or explanations that you do not understand.

X You may not use Generative AI tools during exams.

X You may not use AI tools to fabricate results, citations, experiments, or references.

X You may not use AI tools to disguise plagiarism or copy another student's work.

Students remain fully responsible for everything they submit. You should be able to explain, justify, modify, and debug any submitted code, derivation, result, or written answer.

For homework and project submissions, students should briefly acknowledge meaningful use of Generative AI tools when applicable. For example: "I used ChatGPT to help debug a C/C++ error. I reviewed, modified, and tested the final code myself."

Misuse of Generative AI tools may be treated as an academic integrity issue. The goal is not to ban these tools, but to ensure that they are used responsibly, transparently, and in a way that supports genuine learning.

9. A note for auditing students

1. Auditing students are not required to take the exam or participate in the group projects.
2. Auditing students have to complete at least three practical assignments.

Restricted